

Universidad Autónoma de Santo Domingo

PRIMADA DE AMÉRICA / Fundada el 28 de octubre de 1538

Presentación

Facultad

Ciencias

Escuela

Escuela de informática

Asignatura

Lab Lenguaje de Programación III (INF 5170)

Tema

MVC: origen, evolución y uso actual en aplicaciones web

Estudiante

Cristopher Santiago Helena Perez

Matrícula

100658882

Profesor/a

Radhames Silverio Gónzalez

Fecha

02/11/2026

Indice

Introducción 3

1. ¿Qué es el patrón MVC? 4

2. ¿Cuál es el origen del modelo MVC y para qué fue creado? 4

3. Explicación de cada componente del modelo: Modelo, Vista y Controlador 4

4. Ventajas y desventajas del uso de MVC 5

Ventajas: 5

Desventajas: 5

5. ¿Por qué MVC sigue siendo importante en la programación actual? 5

6. ¿Cómo se usa MVC hoy en día en frameworks modernos? 6

7. Comparar el MVC tradicional con la forma en que se aplica actualmente en frameworks modernos 6

8. Ejemplo sencillo donde se identifique claramente el Modelo, la Vista y el Controlador 6

9. Conclusión personal sobre la importancia de comprender MVC antes de trabajar con frameworks modernos 7

Referencias Bibliográficas (Formato APA 7.ª Edición) 7

Introducción

En la historia del desarrollo de software, uno de los mayores desafíos a los que se han enfrentado los ingenieros es la gestión de la complejidad en sistemas grandes. En los inicios de la computación interactiva, era común que la lógica de la interfaz de usuario, las reglas del negocio y el acceso a los datos estuvieran fuertemente acoplados en un solo bloque de código monolítico. Este diseño, conocido popularmente como *Spaghetti Code*, volvía el software extremadamente difícil de mantener, probar, escalar o modificar, ya que cualquier cambio visual en la pantalla podía romper las operaciones internas del sistema o corromper la base de datos.

Para resolver este problema estructural y lograr una separación de incumbencias (*Separation of Concerns*), a finales de la década de 1970 se introdujo uno de los patrones de diseño arquitectónico más influyentes de la historia de la informática: el **Modelo Vista Controlador (MVC)**. Concebido originalmente para interfaces gráficas de usuario en estaciones de trabajo monousuario, MVC demostró ser tan robusto, intuitivo y adaptable que trascendió su época original, convirtiéndose en el cimiento conceptual sobre el cual se construyeron los sistemas web tradicionales y los frameworks empresariales más potentes de la actualidad. Comprender la filosofía de MVC es indispensable para cualquier desarrollador, ya que permite asimilar con naturalidad el funcionamiento interno de las herramientas tecnológicas modernas.

1. ¿Qué es el patrón MVC?

El **Modelo Vista Controlador (MVC)** es un patrón de arquitectura de software que separa una aplicación interactiva en tres componentes interconectados y lógicamente independientes:

- **El Modelo:** Gestiona los datos, la lógica de negocio y las reglas de almacenamiento de la aplicación.
- **La Vista:** Representa la interfaz visual y la forma en que la información es presentada al usuario.
- **El Controlador:** Actúa como el intermediario que procesa las solicitudes del usuario, interactúa con el Modelo y selecciona la Vista adecuada para responder.

El propósito principal de esta separación es evitar que los cambios en la interfaz gráfica afecten a la lógica de los datos, y viceversa, permitiendo que múltiples desarrolladores trabajen en paralelo sobre diferentes capas de una misma aplicación.

2. ¿Cuál es el origen del modelo MVC y para qué fue creado?

El patrón MVC fue concebido originalmente en **1978 por Trygve Reenskaug** mientras trabajaba en el centro de investigación *XEROX PARC* para el lenguaje de programación Smalltalk.

- **Propósito Inicial:** En esa época, las computadoras comenzaban a transicionar hacia entornos interactivos con interfaces gráficas basadas en pantallas y ratones. Reenskaug ideó MVC para resolver un problema específico: permitir que los usuarios visualizaran y editaran los mismos datos de diferentes maneras simultáneamente (por ejemplo, ver una tabla numérica de ventas y, al mismo tiempo, un gráfico de barras actualizado en tiempo real). El patrón se diseñó para conectar de forma desacoplada la información abstracta (*Model*) con su representación visual (*View*) mediante un componente de control de eventos (*Controller*).

3. Explicación de cada componente del modelo: Modelo, Vista y Controlador

- **Modelo (Model):** Es el corazón de la aplicación. Contiene los datos puros, las estructuras de información, las conexiones con las bases de datos y la lógica de negocio estricta (por ejemplo, calcular promedios, validar restricciones de notas o procesar transacciones). El Modelo es completamente independiente de cómo se muestran los datos en la pantalla.

- **Vista (Vista / View):** Es la capa de presentación visual. Se encarga de mostrar al usuario la información proveniente del Modelo en un formato comprensible (páginas HTML, interfaces gráficas de escritorio, aplicaciones móviles). La Vista no toma decisiones lógicas ni calcula datos; su única función es renderizar la interfaz y capturar las acciones del usuario.
- **Controlador (Controller):** Funciona como el director de orquesta o intermediario. Recibe las peticiones enviadas por el usuario a través de la Vista (por ejemplo, hacer clic en un botón de guardar), interpreta dichas acciones, solicita al Modelo que busque o modifique los datos necesarios, y finalmente decide qué Vista debe mostrarse como respuesta.

4. Ventajas y desventajas del uso de MVC

Ventajas:

- **Separación de Incumbencias:** Permite que la lógica de negocio, la interfaz y el flujo de control estén totalmente aislados, facilitando un código limpio y ordenado.
- **Desarrollo Paralelo:** Varios programadores pueden trabajar simultáneamente en distintas áreas (uno diseña la interfaz visual en la Vista, otro optimiza las consultas en el Modelo y otro gestiona el flujo en el Controlador).
- **Mantenibilidad y Escalabilidad:** Modificar el diseño visual de una página web no requiere alterar las reglas lógicas ni la base de datos subyacente.
- **Facilidad de Pruebas (*Testing*):** La lógica de negocio contenida en el Modelo se puede probar de manera automatizada sin necesidad de levantar una interfaz gráfica en el navegador.

Desventajas:

- **Curva de Aprendizaje Elevada:** Para proyectos muy pequeños o simples, implementar MVC puede añadir una complejidad innecesaria y un exceso de archivos iniciales (*boilerplate*).
- **Sobrecarga de Comunicación:** En aplicaciones mal diseñadas, el flujo excesivo de llamadas entre el Controlador, el Modelo y la Vista puede generar código redundante y acoplamiento indeseado.

5. ¿Por qué MVC sigue siendo importante en la programación actual?

Aunque el patrón nació hace más de cuatro décadas, MVC sigue siendo vital porque **sus principios fundamentales de diseño y modularidad continúan vigentes**. Ningún software moderno de gran escala agrupa todo su código en un solo archivo; todos los patrones arquitectónicos actuales (como MVVM en aplicaciones móviles, MVP, o la arquitectura

limpia) son evoluciones directas o adaptaciones del concepto original de separación de capas que introdujo MVC. Comprender MVC otorga al programador la madurez mental necesaria para entender cómo fluyen los datos y las peticiones en cualquier plataforma de software.

6. ¿Cómo se usa MVC hoy en día en frameworks modernos?

En el desarrollo web contemporáneo, la arquitectura MVC se ejecuta principalmente en el **servidor (Server-Side MVC)** o adaptada a arquitecturas híbridas. Cuando un usuario escribe una URL o envía un formulario, el framework intercepta la petición mediante un enrutador (*Router*), ejecuta el **Controlador** correspondiente, este interactúa con la base de datos a través del **Modelo** (usualmente utilizando un ORM como Hibernate, Eloquent o Entity Framework), y finalmente inyecta los datos resultantes en una plantilla HTML para renderizar la **Vista** de vuelta al navegador.

7. Comparar el MVC tradicional con la forma en que se aplica actualmente en frameworks modernos

Dimensión	**MVC Tradicional (Smalltalk / Escritorio)**	**Frameworks Web Modernos (Laravel, Spring MVC, ASP.NET Core)**
Actualización de Datos	El Modelo notificaba directamente a la Vista mediante el patrón *Observer* para cambios en tiempo real.	Ciclo HTTP unidireccional por petición y respuesta (*Request-Response*). El cliente solicita, el servidor procesa y responde.
Entorno de Ejecución	Aplicaciones monolíticas de escritorio locales.	Arquitecturas web distribuidas, servidores backend y APIs RESTful.
Ecosistema de Componentes	Clases e instancias en memoria de un solo lenguaje.	Enrutadores avanzados, sistemas de plantillas dinámicas, validadores automáticos y ORMs (Mapeo Objeto-Relacional).

8. Ejemplo sencillo donde se identifique claramente el Modelo, la Vista y el Controlador

Tomemos como referencia el sistema de gestión de alumnos de nuestra práctica académica:

- **Modelo:** Representa la estructura de datos del estudiante (matrícula, nombre, calificaciones de parciales y estatus de aprobación) y la lógica para calcular la nota total y verificar si está aprobado.

- **Vista:** El archivo HTML con el formulario web de registro y la tabla interactiva donde se consultan los datos en pantalla.
- **Controlador:** El script o función de backend que recibe los datos enviados desde el formulario HTML, valida que las notas no superen los límites permitidos, ordena al Modelo guardar los cambios en la base de datos y devuelve la respuesta al cliente.

9. Conclusión personal sobre la importancia de comprender MVC antes de trabajar con frameworks modernos

Comprender a profundidad el patrón Modelo Vista Controlador es un requisito indispensable y un rito de paso necesario para cualquier desarrollador que aspire a dominar tecnologías avanzadas. Los frameworks modernos (como Laravel en PHP, Spring Boot en Java, o ASP.NET Core) automatizan gran parte de la estructura visual y de enrutamiento mediante magia interna y convenciones preestablecidas; sin embargo, si un programador no entiende los conceptos fundamentales de cómo se separan los datos, la interfaz y la lógica de control, se enfrentará a una barrera infranqueable al intentar depurar errores complejos o diseñar arquitecturas limpias y escalables. MVC no es solo una estructura de carpetas, es la base mental de la ingeniería de software moderna orientada al orden y el modularidad.

Referencias Bibliográficas (Formato APA 7.ª Edición)

- **Reenskaug, T.** (1979). *Models-Views-Controllers*. Xerox PARC Technical Memorandum. Recuperado el 2 de noviembre de 2026, de http://heim.ifi.uio.no/~trygver/themes/mvc/mvc-index.html
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- **Mozilla Developer Network (MDN).** (2025). *MVC architecture overview*. Mozilla Contributors. Recuperado el 2 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Glossary/MVC
- **Microsoft Learn.** (2024). *ASP.NET Core MVC overview*. Microsoft Documentation. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/es-es/aspnet/core/mvc/overview